

Folder Structure Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.1 (Updated: Frontend switched to JavaScript, no TypeScript)

Status: Final for Implementation **Based On:** System Architecture v1.0, Database Design v1.0, Tech Stack Justification v1.0

1. Purpose

This document defines the complete, production-level repository structure for SentinelCut. It reflects the layered architecture (API layer, service layer, repository layer, AI pipeline) and the FastAPI + Next.js + PostgreSQL/Supabase stack established in prior documents. The structure is designed for a **monorepo** — a single repository containing frontend, backend, AI pipeline, and shared documentation — which is the right choice for a small academic team needing simple version control and coordinated deployments across tightly coupled layers.

2. Top-Level Repository Structure

```
sentinelcut/
├── frontend/           # Next.js web application
├── backend/           # FastAPI backend service
├── ai-pipeline/       # AI/ML processing modules (used by backend
workers)
├── database/          # Migrations, seed data, schema docs
├── infra/             # Deployment configs, Docker, CI/CD
├── docs/              # All project documentation (PRD, SRS,
etc.)
├── scripts/          # Utility/dev scripts
├── tests/             # Cross-cutting/integration/E2E tests
├── .github/           # GitHub Actions workflows
├── .env.example       # Sample environment variables (no secrets)
├── docker-compose.yml # Local dev orchestration
├── README.md
├── CHANGELOG.md
├── CONTRIBUTING.md
├── LICENSE
└── .gitignore
```

3. Frontend Structure (`frontend/`)

Update (v1.1): Per project preference, the frontend uses **Next.js with JavaScript** (not TypeScript). All `.tsx`/`.ts` files below are `.jsx`/`.js`, there is no `tsconfig.json`, and type safety is handled at runtime via **PropTypes** (for component props) and **Zod** (for API/form validation) instead of compile-time types. This is a fully supported, production-viable Next.js configuration — plenty of real-world Next.js apps ship in JS-only mode — and it removes a layer of tooling complexity, which is a reasonable trade-off for a small team optimizing for comfort and delivery speed over static type-checking.


```

UI
|   |   |— dashboard/           # Recent files, credits widget,
storage widget
|   |   |— timeline-editor/     # Transcript view, waveform,
detection cards
|   |   |   |— TranscriptView.jsx
|   |   |   |— DetectionHighlight.jsx
|   |   |   |— PlaybackControls.jsx
|   |   |   |— EditActionsToolbar.jsx
|   |   |— admin/               # Admin-specific tables/forms
|   |   |   |— shared/          # Navbar, Footer, Sidebar, Toasts
|   |— hooks/
|   |   |— useAuth.js
|   |   |— useFileUpload.js
|   |   |— useJobStatus.js      # WebSocket/polling hook
|   |   |   |— useTimelineEditor.js
|   |— lib/
|   |   |— api-client.js        # Centralized fetch/axios wrapper
|   |   |— supabase-client.js
|   |   |— constants.js
|   |   |   |— validators.js    # Client-side schema validation
(zod)
|   |— store/                   # Global state (Zustand/Redux, if
used)
|   |   |— authStore.js
|   |   |   |— editorStore.js
|   |— schemas/                 # Zod schemas (replaces TS types/
folder)
|   |   |— fileSchema.js
|   |   |— detectionSchema.js
|   |   |— userSchema.js
|   |   |   |— apiSchema.js
|   |   |— styles/
|   |       |— globals.css
|— tests/
|   |— unit/
|   |— component/
|   |   |— e2e/                 # Playwright/Cypress specs
|— .eslintrc.json
|— .prettierrc
|— jsconfig.json                # Path aliases (e.g.,
"@/components/*") without TS
|— next.config.js
|— tailwind.config.js

```

Rationale: Route groups ((marketing), (auth), (app), (admin)) map directly to the User Journeys defined in the PRD and allow independent layouts (e.g., admin has its own sidebar) without URL prefix pollution. The (timeline-editor/) component folder is deliberately isolated given it's flagged as the most technically demanding UI piece in the System Architecture document.

Notes on the JS-only setup:

- (jsconfig.json) replaces (tsconfig.json) — it still gives you clean import aliases (e.g., (import Button from '@components/ui/Button')) and enables editor IntelliSense/autocomplete in VS Code without requiring TypeScript itself.
 - The former (types/) folder is replaced by (schemas/), containing **Zod** schemas — these serve double duty as runtime validation (for forms and API responses) and as the de facto documentation of data shapes, which is what the TypeScript types were mainly doing anyway in a project this size.
 - Component prop validation (optional but recommended) can be added incrementally via the (prop-types) package on any component where it adds clarity, without an all-or-nothing TypeScript migration.
 - If the team later wants incremental type safety without a full rewrite, Next.js supports JSDoc-based type checking ((// @ts-check) comments) as a lightweight middle ground — noted here for future reference only, not part of the current plan.
-

4. Backend Structure (`backend/`)

```
backend/
├─ app/
│   │   └─ main.py                # FastAPI app entrypoint
│   │   └─ config.py             # Settings (env-based, Pydantic
Settings)
│   │   └─ dependencies.py        # Shared FastAPI dependencies (auth,
db session)
│   │   └─ api/
│   │       │   └─ v1/
│   │       │       │   └─ routes/
│   │       │       │       │   └─ auth_routes.py
│   │       │       │       │   └─ file_routes.py
│   │       │       │       │   └─ detection_routes.py
│   │       │       │       │   └─ timeline_routes.py
│   │       │       │       │   └─ export_routes.py
│   │       │       │       │   └─ dashboard_routes.py
│   │       │       │       │   └─ admin_routes.py
│   │       │       │       │   └─ preset_routes.py
│   │       │       │       │   └─ feedback_routes.py
│   │       │       │       │   └─ websocket_routes.py
│   │       │       │       └─ router.py    # Aggregates all v1 routers
│   │       │       └─ deps/
│   │       │           │   └─ auth_deps.py    # JWT validation dependency
│   │       │           │   └─ rbac_deps.py    # Role-check dependency
│   │       │   └─ schemas/                # Pydantic request/response models
│   │       │       │   └─ auth_schema.py
│   │       │       │   └─ file_schema.py
│   │       │       │   └─ detection_schema.py
│   │       │       │   └─ job_schema.py
│   │       │       │   └─ preset_schema.py
│   │       │       │   └─ admin_schema.py
│   │       │   └─ models/                # SQLAlchemy ORM models (mirrors
Database Design doc)
│   │       │       │   └─ user.py
│   │       │       │   └─ guest_session.py
│   │       │       │   └─ file.py
│   │       │       │   └─ detection.py
│   │       │       │   └─ detection_config.py
│   │       │       │   └─ job.py
│   │       │       │   └─ job_log.py
│   │       │       │   └─ preset.py
│   │       │       │   └─ profanity_dictionary.py
│   │       │       └─ export.py
```

```

| | | └─ credits_ledger.py
| | | └─ admin_log.py
| | | └─ feedback.py
| | └─ repositories/                                # Repository Pattern (data access
layer)
| | | └─ base_repository.py                        # Generic CRUD base class
| | | └─ user_repository.py
| | | └─ file_repository.py
| | | └─ detection_repository.py
| | | └─ job_repository.py
| | | └─ preset_repository.py
| | | └─ dictionary_repository.py
| | | └─ admin_log_repository.py
| | └─ services/                                    # Business logic layer
| | | └─ auth_service.py
| | | └─ file_service.py
| | | └─ job_orchestrator_service.py
| | | └─ detection_config_service.py
| | | └─ timeline_edit_service.py
| | | └─ export_service.py
| | | └─ admin_service.py
| | | └─ credits_service.py
| | └─ core/
| | | └─ security.py                                # JWT encode/decode, password
hashing
| | | └─ exceptions.py                              # Custom exception classes
| | | └─ logging_config.py
| | | └─ constants.py
| | └─ db/
| | | └─ session.py                                # SQLAlchemy engine/session setup
| | | └─ base.py                                    # Declarative base import aggregator
| | └─ workers/
| | | └─ celery_app.py                              # Celery/RQ app configuration
| | | └─ tasks/
| | | | └─ processing_task.py                      # Orchestrates the full AI pipeline
call
| | | | └─ regeneration_task.py
| | | | └─ cleanup_task.py                        # Guest session/expired file purging
| | | | └─ status_publisher.py                    # Publishes job status to Redis for
WebSocket relay
└─ tests/
| | └─ unit/
| | | └─ test_services/
| | | └─ test_repositories/
| └─ integration/

```

```
| | | └─ test_api_routes/
| | | └─ test_db/
| | └─ conftest.py
└─ alembic/                                # DB migrations (see database/ for
canonical schema docs)
| | └─ versions/
| | └─ env.py
└─ requirements.txt
└─ requirements-dev.txt
└─ Dockerfile
└─ pytest.ini
└─ .flake8
└─ README.md
```

Rationale: The `repositories/` and `services/` split directly implements the Repository Pattern and layered service architecture mandated in the Coding Standards (Product Vision brief) and reflected in the System Architecture component diagram.

`workers/tasks/` maps one-to-one to the async job types defined in the `jobs.job_type` enum in the Database Design document.

5. AI Pipeline Structure (`ai-pipeline/`)

```
ai-pipeline/
├── sentinelcut_ai/
│   ├── __init__.py
│   ├── asr/
│   │   ├── __init__.py
│   │   ├── speech_recognition_service.py # Interface (abstract base)
│   │   ├── faster_whisper_engine.py      # Concrete implementation
│   │   └── forced_alignment.py           # Word-level timestamp alignment
│   ├── language_detection/
│   │   ├── __init__.py
│   │   └── language_detector.py
│   ├── profanity_detection/
│   │   ├── __init__.py
│   │   ├── profanity_detection_service.py # Interface (abstract base)
│   │   └── normalizer.py                 # Repeated-char, leetspeak, Unicode
normalization
│   ├── dictionary_matcher.py             # Dictionary + regex layer
│   └── context_scorer.py                 # Embedding/IndicBERT-based scoring
layer
│   └── confidence_aggregator.py
├── audio_processing/
│   ├── __init__.py
│   ├── audio_extractor.py                # FFmpeg audio extraction wrapper
│   └── audio_editor.py                   # PyDub beep/mute/custom-sound
application
│   ├── media_muxer.py                    # FFmpeg re-mux wrapper
│   ├── pipeline/
│   │   ├── __init__.py
│   │   └── processing_pipeline.py        # Orchestrates full pipeline
(Section 5 of Architecture doc)
│   ├── pipeline_config.py
│   ├── data/
│   │   ├── dictionaries/
│   │   │   ├── english_profanity.json
│   │   │   ├── hindi_profanity.json
│   │   │   └── hinglish_slang.json
│   │   ├── embeddings/
│   │   │   └── seed_profanity_examples.json
│   └── utils/
│       ├── file_utils.py
│       ├── timing_utils.py
│       └── logging_utils.py
└── notebooks/                          # Experimentation (model
```

```
evaluation, accuracy testing)
|   └─ asr_benchmark.ipynb
|   └─ profanity_detection_eval.ipynb
└─ tests/
    └─ test_normalizer.py
    └─ test_dictionary_matcher.py
    └─ test_context_scorer.py
    └─ test_pipeline_integration.py
└─ requirements.txt
└─ Dockerfile
└─ README.md
```

Rationale: This module is structured to be importable directly by `backend/app/workers/tasks/processing_task.py`, keeping AI logic testable and swappable independent of the API layer — directly implementing the "Extensibility Points" defined in the System Architecture document (Section 7.3).

6. Database Structure (`database/`)

```
database/
└─ migrations/                                # Mirrors backend/alembic if kept
separate for clarity
    └─ (versioned migration files)
└─ seeds/
    └─ seed_profanity_dictionary.py
    └─ seed_dev_users.py
└─ schema/
    └─ er_diagram.md                          # Reference copy of ER diagram
    └─ schema_reference.sql                  # Human-readable full DDL reference
└─ README.md
```

Note: In practice, Alembic migrations physically live under `backend/alembic/` (Section 4) since they're tightly coupled to the SQLAlchemy models; `database/` serves as the canonical, framework-agnostic reference copy of the schema for documentation and onboarding purposes.

7. Infrastructure & Deployment Structure (**infra/**)

```
infra/
├── docker/
│   ├── frontend.Dockerfile
│   ├── backend.Dockerfile
│   └── worker.Dockerfile
├── docker-compose.dev.yml          # Local dev: backend + worker +
redis + postgres
├── vercel.json                    # Frontend deployment config
├── render.yaml                   # Backend/worker deployment config
(Render Blueprint)
├── supabase/
│   ├── config.toml
│   └── rls_policies.sql          # Row-Level Security policy
definitions
└── env/
    ├── .env.staging.example
    └── .env.production.example
```

8. CI/CD Structure (**.github/**)

```
.github/
├── workflows/
│   ├── frontend-ci.yml          # Lint, type-check, build, test on
PR
│   ├── backend-ci.yml          # Lint, test, coverage on PR
│   └── ai-pipeline-ci.yml       # Unit tests for detection/ASR
modules
│   └── deploy.yml              # Deploy on merge to main
├── ISSUE_TEMPLATE/
│   ├── bug_report.md
│   └── feature_request.md
└── PULL_REQUEST_TEMPLATE.md
```

9. Documentation Structure (`docs/`)

```
docs/  
├─ 01_Product_Vision.md  
├─ 02_PRD.md  
├─ 03_SRS.md  
├─ 04_Tech_Stack_Justification.md  
├─ 05_System_Architecture.md  
├─ 06_Database_Design.md  
├─ 07_Folder_Structure.md  
├─ 08_API_Design.md  
├─ 09_AI_Pipeline.md  
├─ 10_UIUX_Design.md  
├─ 11_Development_Roadmap.md  
├─ 12_Implementation_Notes.md  
├─ 13_Testing/  
│   ├─ Test_Plan.md  
│   ├─ Test_Cases.md  
│   └─ QA_Checklist.md  
├─ 14_Deployment_Guide.md  
├─ 15_Future_Scope.md  
├─ manuals/  
│   ├─ User_Manual.md  
│   ├─ Admin_Manual.md  
│   └─ Maintenance_Guide.md  
├─ academic/  
│   ├─ College_Report.md  
│   ├─ IEEE_Research_Paper.md  
│   ├─ Project_Synopsis.md  
│   ├─ Project_Proposal.md  
│   ├─ Weekly_Progress_Reports/  
│   ├─ Final_Viva_Questions.md  
│   ├─ Demo_Script.md  
│   ├─ Poster_Content.md  
│   └─ Presentation_Outline.md  
├─ standards/  
│   ├─ Coding_Standards.md  
│   ├─ Git_Workflow.md  
│   ├─ Security_Guidelines.md  
│   └─ Versioning_Strategy.md  
├─ legal/  
│   ├─ Privacy_Policy.md  
│   └─ Terms_and_Conditions.md  
└─ misc/  
    └─ Resume_Description.md
```

```
└─ LinkedIn_Description.md
└─ GitHub_Description.md
└─ Release_Notes.md
└─ Roadmap.md
```

Rationale: This maps one-to-one to the full deliverables list from the original project brief, ensuring nothing from the master scope is structurally lost or homeless.

10. Scripts (`scripts/`)

```
scripts/
└─ setup_dev_env.sh                # One-command local environment
bootstrap
└─ run_migrations.sh
└─ seed_database.sh
└─ generate_test_media.py          # Creates sample audio/video files
with known profanity for testing
└─ benchmark_asr.py               # Runs ASR accuracy benchmarks
against labeled test set
└─ benchmark_detection.py         # Runs profanity detection
precision/recall benchmarks
```

11. Root-Level Configuration Files

File	Purpose
<code>.env.example</code>	Documents all required environment variables without real secrets (DB URL, Supabase keys, Redis URL, JWT secret, email API key)
<code>docker-compose.yml</code>	Orchestrates local development: Postgres, Redis, backend, worker, frontend
<code>README.md</code>	Project overview, setup instructions, links to <code>docs/</code>
<code>CHANGELOG.md</code>	Version history following Keep a Changelog format
<code>CONTRIBUTING.md</code>	Contribution guide referencing <code>docs/standards/Git_Workflow.md</code>
<code>.gitignore</code>	Excludes <code>node_modules/</code> , <code>__pycache__/</code> , <code>.env</code> , media upload test fixtures, etc.

12. Directory-to-Architecture Traceability

Architecture Component (from System Architecture doc)	Folder Location
Next.js Web App	frontend/
FastAPI RESTAPI Service	backend/app/api/
Auth Middleware	backend/app/api/deps/auth_deps.py
WebSocket/Status Service	backend/app/api/v1/routes/websocket_routes.py, backend/app/workers/status_publisher.py
Celery/RQ Worker Pool	backend/app/workers/
SpeechRecognitionService	ai-pipeline/sentinelcut_ai/asr/
ProfanityDetectionService	ai-pipeline/sentinelcut_ai/profanity_detection/
AudioEditingService / MediaMuxService	ai-pipeline/sentinelcut_ai/audio_processing/
Repository Layer	backend/app/repositories/
PostgreSQL Schema	database/schema/, backend/alembic/
Deployment Configs	infra/

13. Next Document

Document 8: API Design — full REST API specification (endpoints, request/response schemas, status codes, WebSocket contract) implementing the routes scaffolded in backend/app/api/v1/routes/ above.

End of Folder Structure Document